



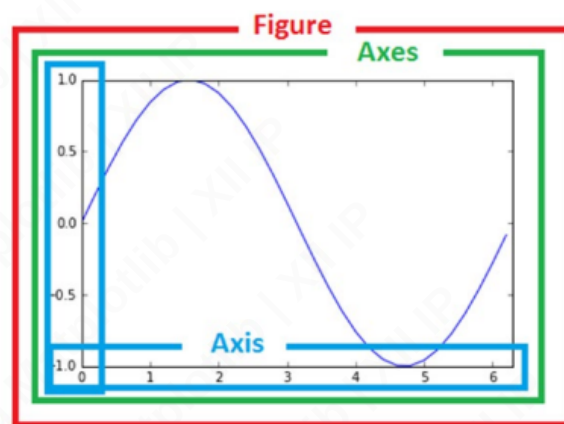
Data Visualization with Matplotlib

► Introduction

When data are shown in the form of pictures, it becomes easy for the user to understand it. So, representing data in the form of pictures or graphs is known as “**data visualization**”.

Data visualization is the discipline of trying to expose the data to understand it by placing it in a visual context. Its main goal is to distil large datasets into visual graphics to allow for easy understanding of complex relationships within the data.

It represents (patterns, trends, correlations etc.) in data and thereby helps decision makers to understand the meaning of data for making decision in business.



- Matplotlib is a python library which provides many interfaces and function to present data in 2D graphics. We can say, Matplotlib is a high-quality plotting library of Python.
- Matplotlib library offers many different collections of sub modules; Pyplot is one such sub module.
- Pyplot is a collection of methods within Matplotlib library which allows user to construct 2D plots easily.

► Types of Charts

Chart Type	Description	Function
Line Chart	Shows trends over time or sequence	plt.plot()
Bar Chart	Displays data in rectangular bars	plt.bar()
Histogram	Shows distribution of continuous data	plt.hist()





Line Plot

A line plot/chart is a graph that shows the frequency of data occurring along a number line.

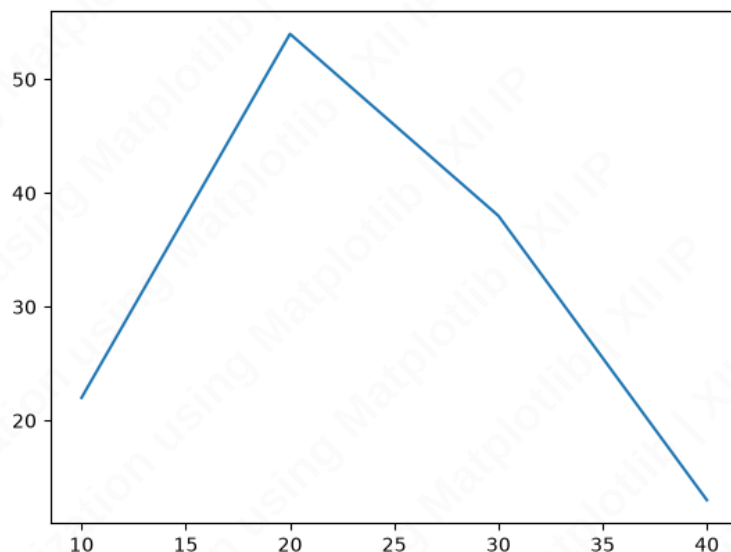
The line plot is represented by a series of datapoints connected with a straight line. Generally, line plots are used to display trends over time. A line plot or line graph can be created using the **plot()** function available in pyplot library. We can, not only just plot a line but we can explicitly define the grid, the x and y axis scale and labels, title and display options etc.

Snippet 1: Basic Line Plot

A first look at plotting two lists of values against each other as a simple line.

```
import matplotlib.pyplot as plt
value_x = [10,20,30,40]
value_y = [22,54,38,13]
```

```
plt.plot(value_x,value_y)
plt.show()
```



What this does: `plt.plot(x, y)` draws a line connecting the (x, y) coordinate pairs in the order given, and `plt.show()` renders the figure. With no styling arguments, Matplotlib uses its default solid blue line and no markers.

Snippet 2: Adding Circle Markers

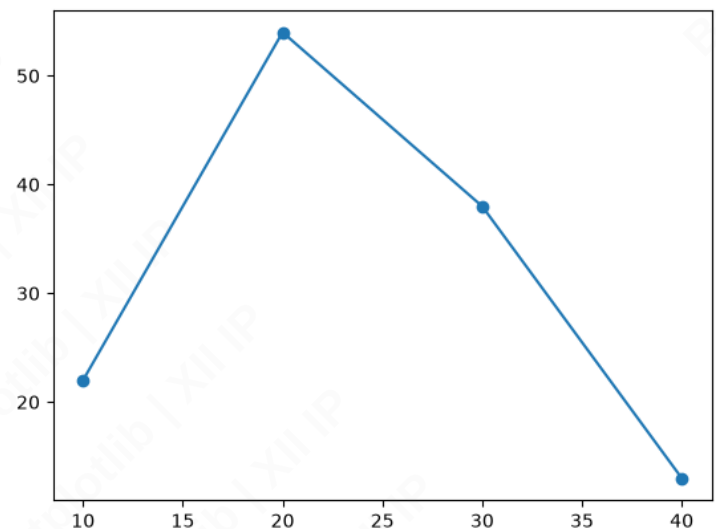
The same line chart, but now each data point is highlighted with a marker.





Data Visualization using Matplotlib

```
plt.plot(value_x,value_y,marker="o")  
plt.show()
```

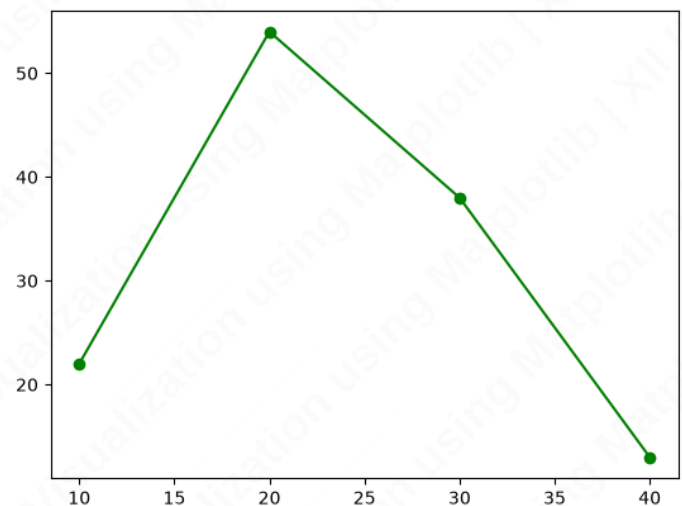


What this does: The marker="o" parameter places a circular marker at every (x, y) point, making individual data points easier to spot in addition to the trend line.

Snippet 3: Changing Line Color

Customising the line color alongside the marker from the previous example.

```
plt.plot(value_x,value_y,marker="o",color="g")  
plt.show()
```



What this does: color="g" sets the line (and by default the marker edge) to green. Matplotlib accepts single-letter color codes (g, r, b, k, ...), full color names, or hex codes.

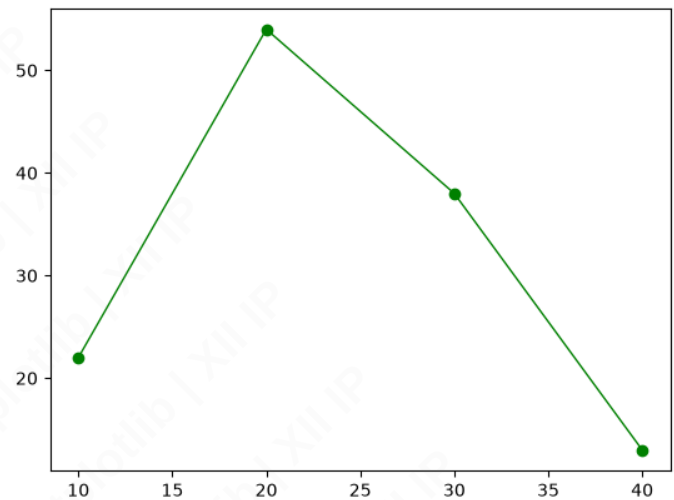




Snippet 4: Controlling Line Width

Adjusting how thick the plotted line appears.

```
plt.plot(value_x,value_y,marker="o",color="g",  
linewidth=1)  
plt.show()
```

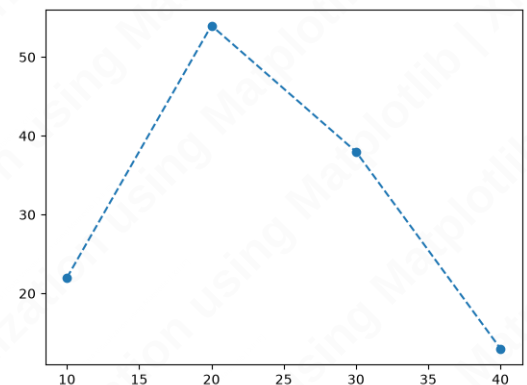


What this does: linewidth (or its alias lw) controls the thickness of the line in points. Here it is set to 1, producing a thinner line than the default width of 1.5.

Snippet 5: Dashed Line Style

Switching from a solid line to a dashed style.

```
plt.plot(value_x,value_y,marker="o",linestyle="dashed")  
plt.show()
```



What this does: linestyle="dashed" (equivalent to the shorthand "--") draws the connecting line as a series of dashes instead of a continuous stroke.

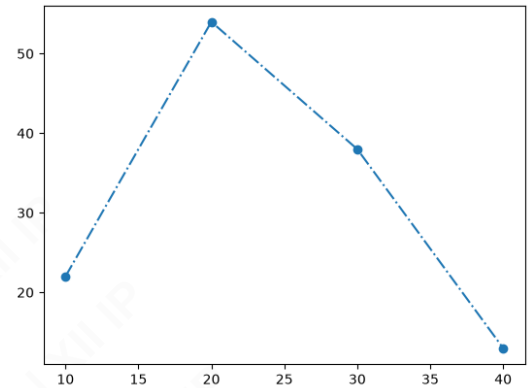




Snippet 6: Dash-Dot Line Style

Using an alternating dash-dot pattern for the line.

```
plt.plot(value_x,value_y,marker="o",linestyle="dashdot")
plt.show()
```

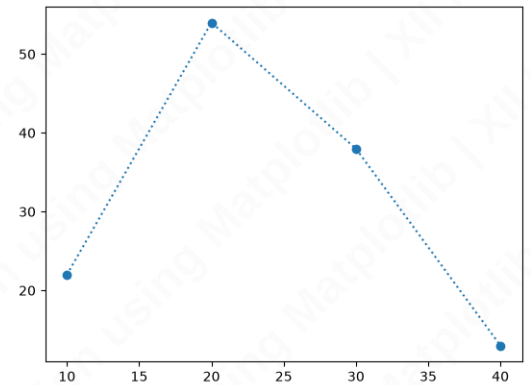


What this does: `linestyle="dashdot"` (shorthand `"-."`) alternates dashes and dots along the line, useful for visually distinguishing multiple series.

Snippet 7: Dotted Line Style

Rendering the connecting line as a dotted pattern.

```
plt.plot(value_x,value_y,marker="o",linestyle="dotted")
plt.show()
```



What this does: `linestyle="dotted"` (shorthand `":"`) draws the line using small dots, giving a lighter visual weight than a solid or dashed line.

Common Marker Styles

Marker	Description	Marker	Description
'.'	Point (pixel)	's'	Square
','	Pixel	'p'	Pentagon
'o'	Circle	'*'	Star
'v'	Triangle down	'h'	Hexagon 1
'^'	Triangle up	'H'	Hexagon 2
'<'	Triangle left	'+'	Plus
'>'	Triangle right	'x'	X
'1'	Tri-down (Y-shaped)	'X'	Filled X





Marker	Description	Marker	Description
'2'	Tri-up (Y-shaped)	'D'	Diamond
'3'	Tri-left (Y-shaped)	'd'	Thin diamond
'4'	Tri-right (Y-shaped)	'\n'	\n
		'_'	Horizontal line

Marker Formatting Options

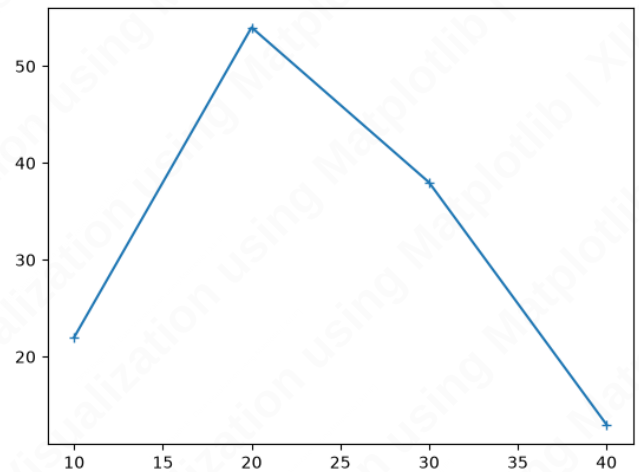
We can customize markers with additional parameters:

Parameter	Purpose	Example
markersize or ms	Marker size	ms=10
markerfacecolor or mfc	Fill color	mfc='red'
markeredgecolor or mec	Border color	mec='black'
markeredgewidth or mew	Border width	mew=2

Snippet 8: Plus-Shaped Markers

Trying a different marker symbol at each data point.

```
plt.plot(value_x,value_y,marker="+")  
plt.show()
```



What this does: marker="+" swaps the marker shape to a plus sign. Matplotlib supports many single-character marker codes (+, x, o, s, d, D, *, and more).

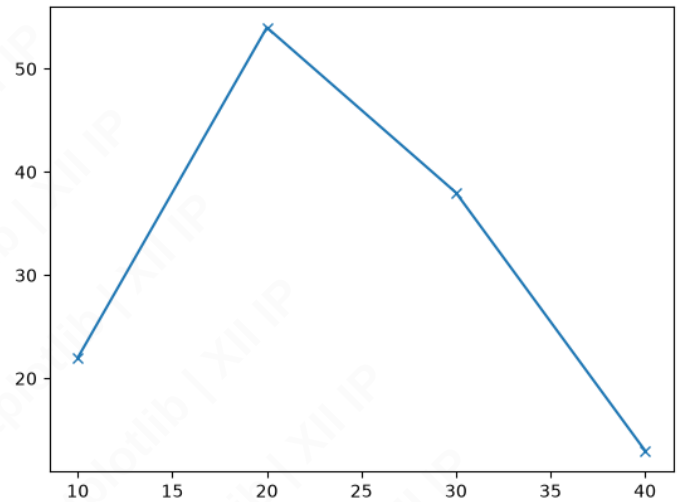




Snippet 9: Cross (X) Markers

Another marker style, this time an X shape.

```
plt.plot(value_x,value_y,marker="x")  
plt.show()
```

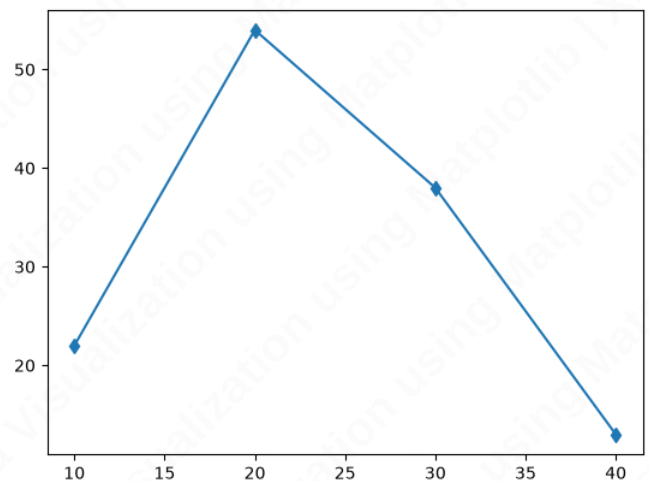


What this does: marker="x" places an X-shaped marker at each point instead of the plus sign used previously.

Snippet 10: Thin Diamond Markers

Using the thin-diamond marker style.

```
plt.plot(value_x,value_y,marker="d")  
plt.show()
```



What this does: marker="d" (lowercase) draws a thin diamond at each data point, narrower than the regular diamond.

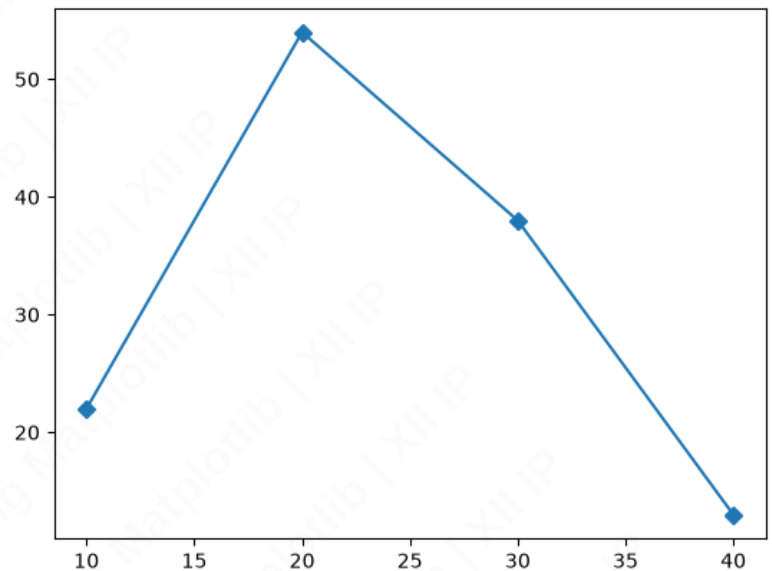




Snippet 11: Diamond Markers

Comparing the thin diamond above to the regular diamond marker.

```
plt.plot(value_x,value_y,marker="D")  
plt.show()
```

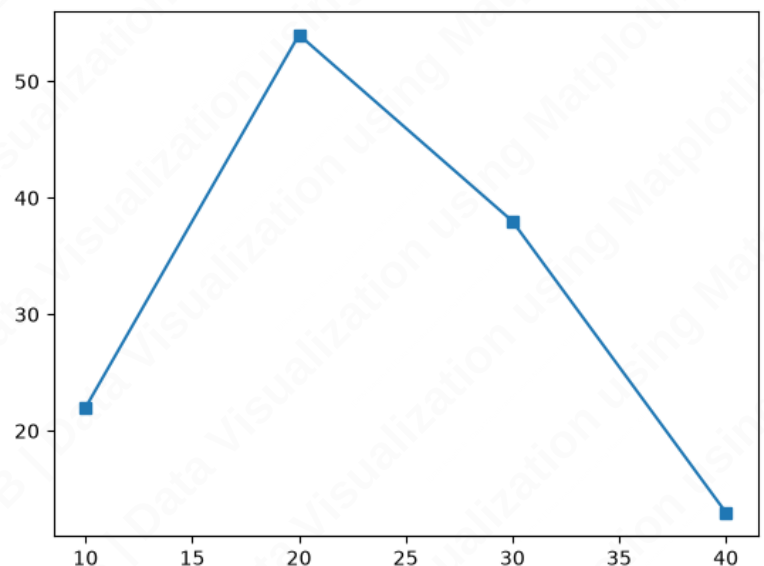


What this does: marker="D" (uppercase) draws a regular, wider diamond shape, as opposed to the thin diamond from marker="d".

Snippet 12: Square Markers

Marking each point with a square instead of a diamond.

```
plt.plot(value_x,value_y,marker="s")  
plt.show()
```



What this does: marker="s" places a square marker at every data point.

Snippet 13: Star Markers

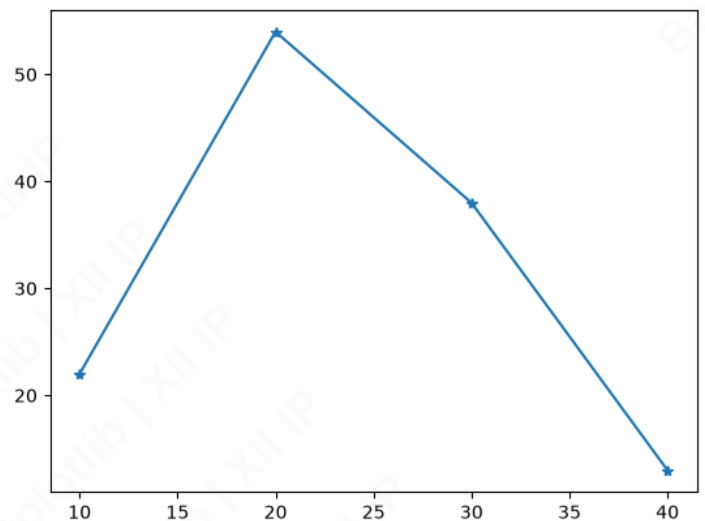
Trying a star-shaped marker for the data points.





Data Visualization using Matplotlib

```
plt.plot(value_x,value_y,marker="*")  
plt.show()
```

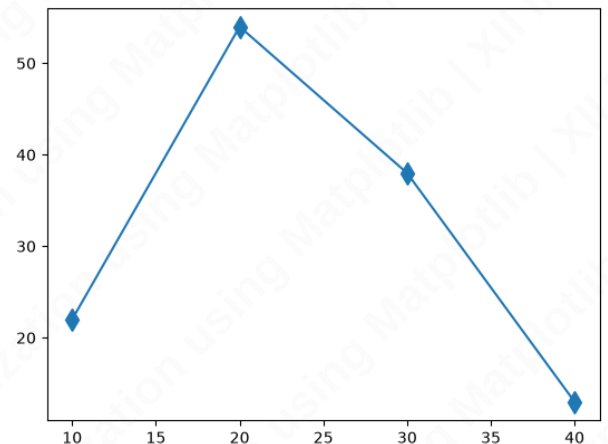


What this does: marker="*" draws a five-pointed star at each point on the line.

Snippet 14: Increasing Marker Size

Making the thin-diamond markers larger and more prominent.

```
plt.plot(value_x,value_y,marker="d",markersize=10)  
plt.show()
```



What this does: markersize sets the marker size in points; increasing it to 10 makes each diamond noticeably bigger than the default size.

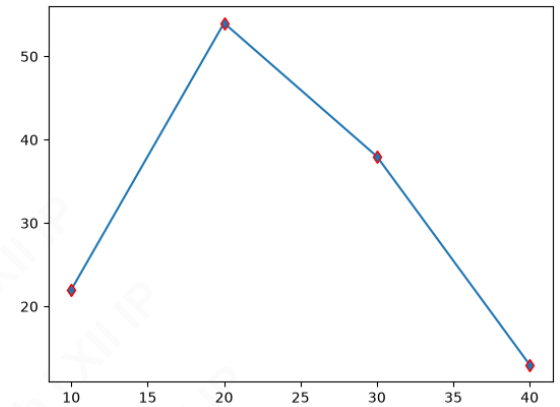




Snippet 15: Marker Edge Color

Highlighting the outline of each marker in a different color.

```
plt.plot(value_x,value_y,marker="d",markeredgcolor="r")  
plt.show()
```

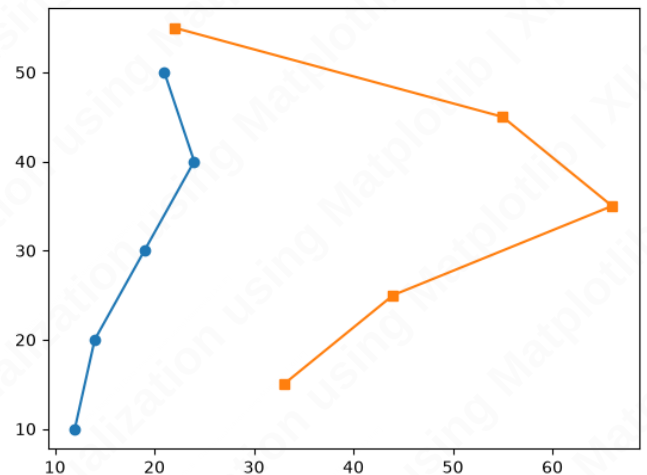


What this does: `markeredgcolor` sets the color of the marker's outline. Here the diamond markers get a red edge while the marker fill and line remain the default blue.

Snippet 16: Plotting Multiple Lines

Drawing two independent lines, from two different pairs of x/y lists, on the same chart.

```
a = [12,14,19,24,21]  
b = [10,20,30,40,50]  
  
x = [33,44,66,55,22]  
y = [15,25,35,45,55]  
  
plt.plot(a,b,marker="o")  
plt.plot(x,y,marker="s")  
  
plt.plot()
```



What this does: Calling `plt.plot()` more than once before `plt.show()` adds each call as a separate line on the same axes. Matplotlib automatically cycles through its default color palette so the two lines are visually distinct.





Snippet 17: Labels, Legend and Title

Making the multi-line chart self-explanatory with axis labels, a legend and a title.

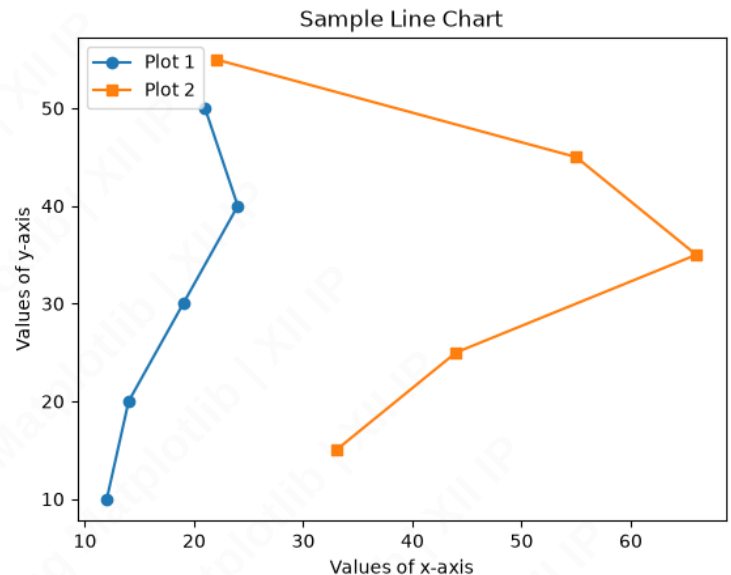
```
a = [12,14,19,24,21]
b = [10,20,30,40,50]

x = [33,44,66,55,22]
y = [15,25,35,45,55]

plt.plot(a,b,marker="o",label="Plot 1")
plt.plot(x,y,marker="s",label="Plot 2")

plt.xlabel("Values of x-axis")
plt.ylabel("Values of y-axis")
plt.legend(loc="upper left")
plt.title("Sample Line Chart")

plt.plot()
```



What this does: label= on each plot() call names that series for the legend. plt.xlabel()/plt.ylabel() caption the axes, plt.title() adds a chart title, and plt.legend(loc="upper left") displays a legend box anchored to the upper-left corner using the labels supplied earlier.

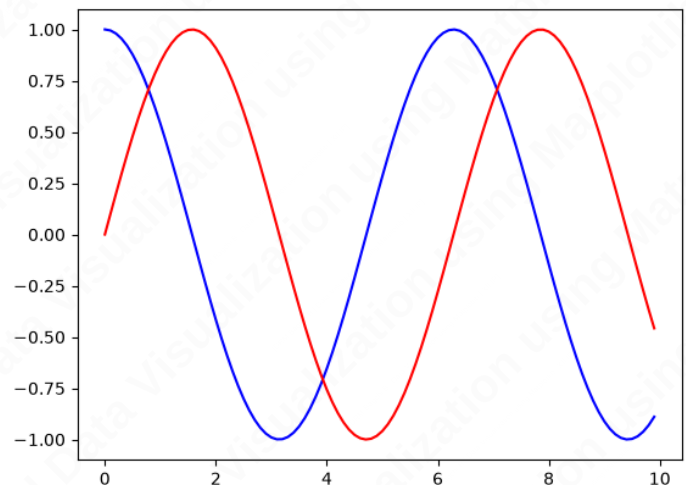
Snippet 18: Plotting Mathematical Functions

Using NumPy to generate data and plotting the cosine and sine functions together.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(0.,10.,0.1)
a = np.cos(x)
b = np.sin(x)

plt.plot(x,a,'b')
plt.plot(x,b,'r')
plt.show()
```



What this does: np.arange(0., 10., 0.1) generates evenly spaced x-values from 0 to 10 in steps of 0.1. np.cos(x) and np.sin(x) compute the cosine and sine at each x-value. The two curves are plotted with shorthand color codes: 'b' for blue (cosine) and 'r' for red (sine).





Working with a Pandas DataFrame

Snippet 19: Creating a Sample DataFrame

Before plotting real records, a small DataFrame of student marks is created to use as the data source for the remaining charts.

```
import pandas as pd

data = {
    'Name'      : ['Amit', 'Bhavna', 'Chintu', 'Deepa', 'Esha',
                  'Faisal', 'Gaurav', 'Hina', 'Ishan', 'Sara', 'Ravi'],
    'Class'     : [11, 12, 11, 11, 12, 11, 12, 11, 12, 12, 11],
    'Section'   : ['A', 'B', 'A', 'C', 'A', 'B', 'A', 'B', 'C', 'A', 'C'],
    'Marks'     : [45, 85, 30, 92, 55, 62, 78, 25, 48, 95, 60],
    'Physics'   : [50, 80, 20, 95, 60, 65, 80, 30, 40, 90, 55],
    'Chemistry' : [40, 85, 35, 90, 50, 70, 75, 20, 55, 92, 65],
    'English'   : [60, 75, 40, 88, 65, 55, 85, 45, 60, 89, 70],
    'Remarks'  : ['Average', 'Good', 'Poor', 'Excellent', 'Average',
                  'Average', 'Good', 'Poor', 'Average', 'Excellent', 'Average']
}

df = pd.DataFrame(data)
```

What this does: A Python dictionary of column-name to value-list pairs is passed to `pd.DataFrame()`, which builds a tabular DataFrame with one row per student and one column per subject/attribute. This `df` is reused as the data source for every chart in the rest of the notebook.

Snippet 20: Viewing the DataFrame

Displaying the DataFrame to confirm the data looks correct before plotting it.

```
print(df)
```

What this does: Simply naming the variable `df` at the end of a Jupyter cell triggers its rich HTML table repr, showing all 11 rows and 8 columns (Name, Class, Section, Marks, Physics, Chemistry, English, Remarks) for a quick visual check of the data.

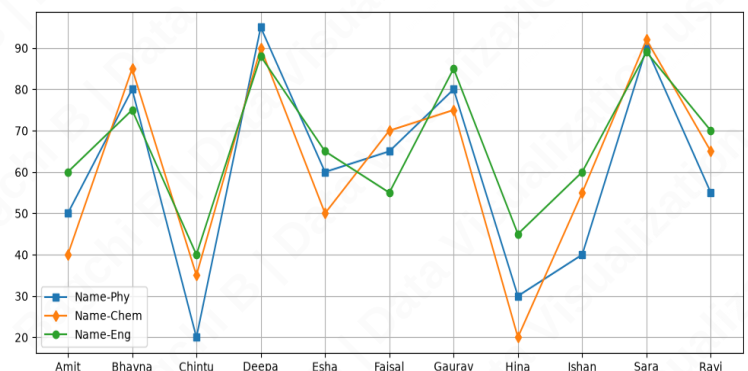
Charting DataFrame Columns

Snippet 21: Marks by Subject Against Student Name

Plotting three subject-mark columns from the DataFrame against student names on a single chart.

```
plt.figure(figsize=(12, 5))

plt.plot(df['Name'], df['Physics'], marker='s', label="Name-Phy")
plt.plot(df['Name'], df['Chemistry'], marker='d', label="Name-Chem")
plt.plot(df['Name'], df['English'], marker='o', label="Name-Eng")
plt.legend()
plt.grid()
plt.show()
```





Data Visualization using Matplotlib

What this does: `plt.figure(figsize=(12, 5))` enlarges the figure so 11 student names fit legibly along the x-axis. Three `plt.plot()` calls draw one line per subject directly against `df['Name']`, each with its own marker and legend label, and `plt.grid()` overlays reference gridlines.

Snippet 22: Using Numeric Positions with Custom Tick Labels

Plotting against numeric positions instead of names directly, then relabelling the x-axis with the student names.

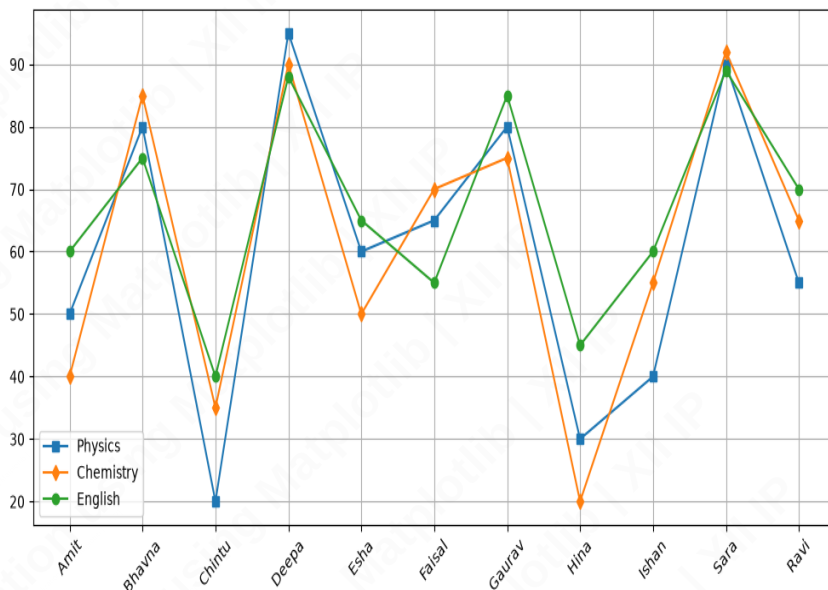
```
x = range(len(df))

plt.figure(figsize=(10, 5))

plt.plot(x, df['Physics'],
marker="s", label="Physics")
plt.plot(x, df['Chemistry'],
marker="d", label="Chemistry")
plt.plot(x, df['English'],
marker="o", label="English")

plt.xticks(x, df['Name'],
rotation=45)

plt.grid()
plt.legend()
plt.tight_layout()
plt.show()
```



What this does: `range(len(df))` creates evenly spaced integer positions (0, 1, 2, ...) used as the x-values so the lines are evenly spread. `plt.xticks(x, df['Name'], rotation=45)` then replaces those numeric tick labels with the actual student names, rotated 45 degrees, and `plt.tight_layout()` trims extra whitespace so the rotated labels are not cut off.

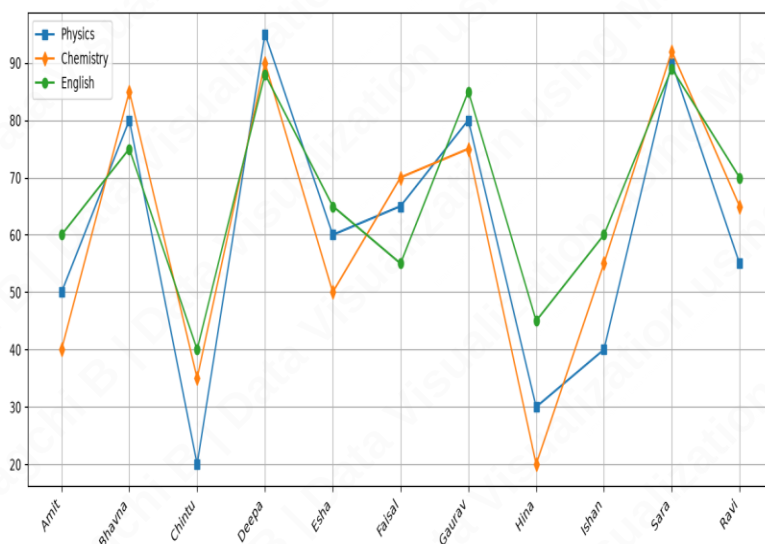
Snippet 23: Rotating and Right-Aligning Name Labels

Revisiting the direct name-based plot, this time rotating and right-aligning the x-axis labels for readability.

```
plt.figure(figsize=(12, 5)) # Increase
plot width

plt.plot(df['Name'], df['Physics'],
marker="s", label="Physics")
plt.plot(df['Name'], df['Chemistry'],
marker="d", label="Chemistry")
plt.plot(df['Name'], df['English'],
marker="o", label="English")

plt.xticks(rotation=45, ha='right') #
Rotate names for better spacing
plt.legend()
plt.grid()
plt.tight_layout() # Prevent labels
from being cut off
plt.show()
```





Data Visualization using Matplotlib

What this does: `plt.xticks(rotation=45, ha='right')` tilts the name labels 45 degrees and horizontally aligns (ha) them to their right edge, which keeps closely spaced names from overlapping compared to plain rotation alone.

Snippet 24: Increasing Horizontal Spacing Between Points

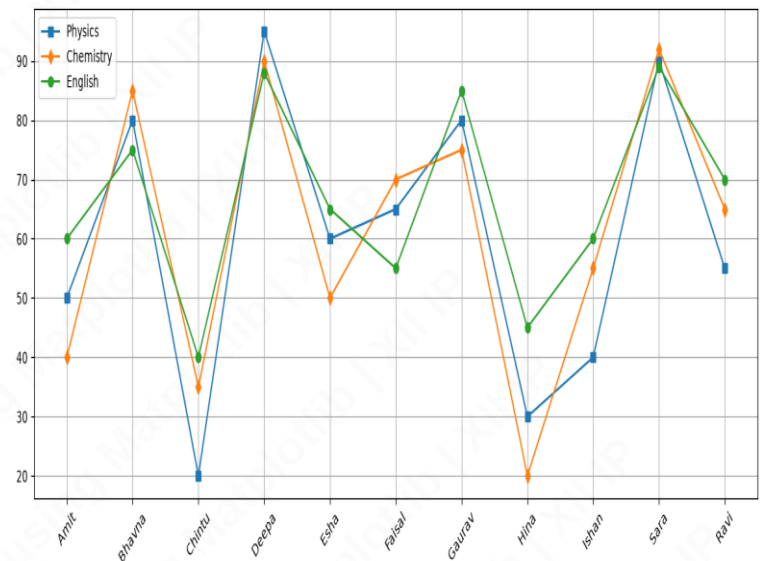
Spreading the data points further apart along the x-axis by scaling the numeric positions.

```
x = np.arange(len(df)) * 2 # Multiply
by 2 to increase spacing

plt.figure(figsize=(12, 5))

plt.plot(x, df['Physics'], marker='s',
label='Physics')
plt.plot(x, df['Chemistry'], marker='d',
label='Chemistry')
plt.plot(x, df['English'], marker='o',
label='English')

plt.xticks(x, df['Name'], rotation=45)
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()
```



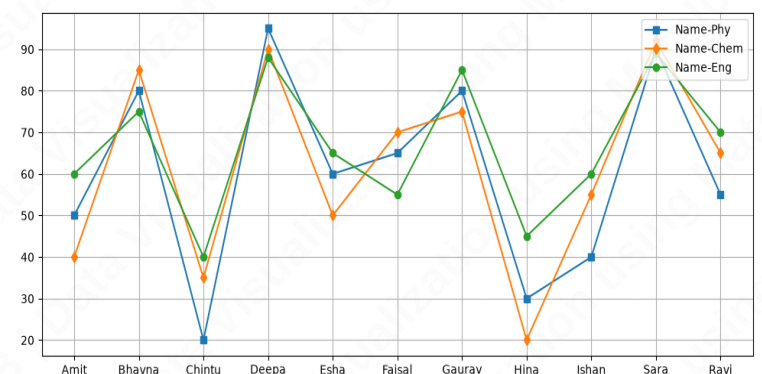
What this does: `np.arange(len(df)) * 2` doubles the spacing between consecutive x positions (0, 2, 4, ...) so points are further apart horizontally, while `plt.xticks(x, df['Name'])` still maps each of those wider positions back to the correct student name.

Snippet 25: Legend Position: Upper Right (loc=1)

Exploring numeric legend location codes, starting with the upper-right corner.

```
plt.figure(figsize=(12, 5))

plt.plot(df['Name'], df['Physics'], marker=
"s", label="Name-Phy")
plt.plot(df['Name'], df['Chemistry'], mark
er="d", label="Name-Chem")
plt.plot(df['Name'], df['English'], marker
="o", label="Name-Eng")
plt.legend(loc=1) # upper right corner
plt.grid()
plt.show()
```



What this does: `plt.legend()` accepts either a string or an integer location code; `loc=1` corresponds to "upper right", pinning the legend box to the top-right corner of the axes regardless of where the data lines fall.



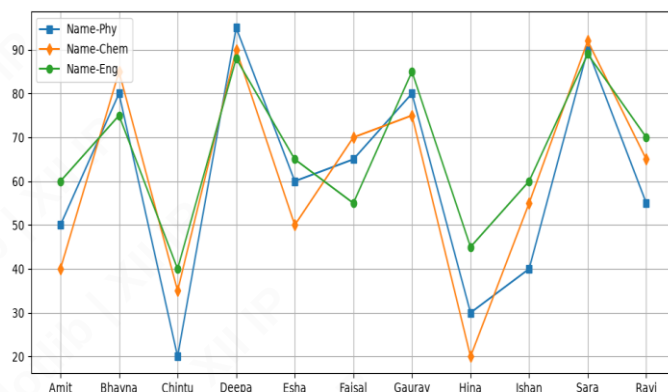


Snippet 26: Legend Position: Upper Left (loc=2)

Moving the legend to the upper-left corner using loc=2.

```
plt.figure(figsize=(12, 5))

plt.plot(df['Name'],df['Physics'],marker="s",label="Name-Phy")
plt.plot(df['Name'],df['Chemistry'],marker="d",label="Name-Chem")
plt.plot(df['Name'],df['English'],marker="o",label="Name-Eng")
plt.legend(loc=2) # upper left corner
plt.grid()
plt.show()
```



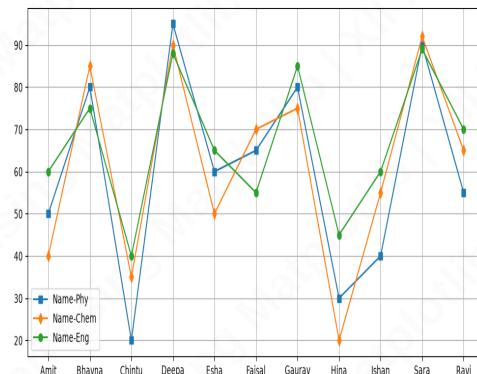
What this does: loc=2 places the legend in the "upper left" corner of the axes, useful here since the data trends upward toward the right, leaving the top-left area empty.

Snippet 27: Legend Position: Lower Left (loc=3)

Trying the lower-left legend position with loc=3.

```
plt.figure(figsize=(12, 5))

plt.plot(df['Name'],df['Physics'],marker="s",label="Name-Phy")
plt.plot(df['Name'],df['Chemistry'],marker="d",label="Name-Chem")
plt.plot(df['Name'],df['English'],marker="o",label="Name-Eng")
plt.legend(loc=3) # Bottom Left corner
plt.grid()
plt.show()
```



What this does: loc=3 anchors the legend to the "lower left" corner. Numeric location codes in Matplotlib run from 0 (best) through 10, each mapping to a specific corner or edge position.



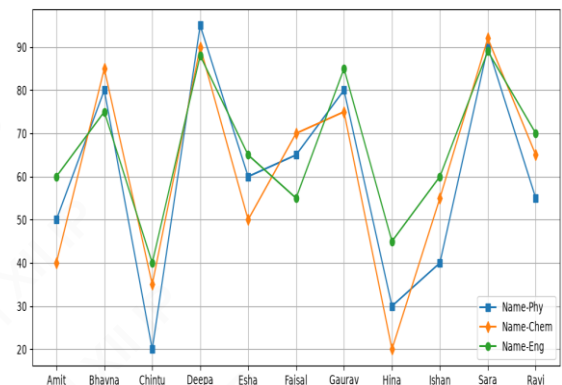


Snippet 28: Legend Position: Lower Right (loc=4)

Placing the legend in the lower-right corner with loc=4.

```
plt.figure(figsize=(12, 5))

plt.plot(df['Name'],df['Physics'],marker="s",label="Name-Phy")
plt.plot(df['Name'],df['Chemistry'],marker="d",label="Name-Chem")
plt.plot(df['Name'],df['English'],marker="o",label="Name-Eng")
plt.legend(loc=4) # bottom right corner
plt.grid()
plt.show()
```



What this does: loc=4 anchors the legend to the "lower right" corner of the plotting area, completing the tour of the four main corner positions.

Snippet 29: Gridlines on the X-Axis Only

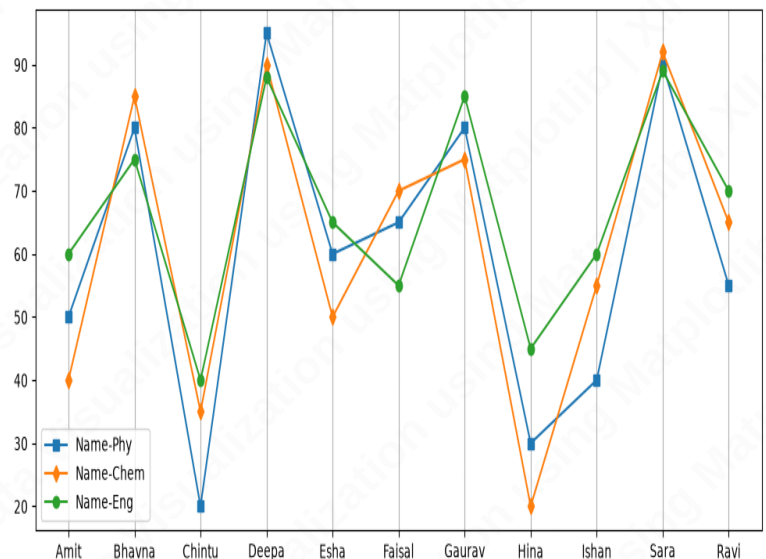
Restricting the grid so it only shows vertical lines along the x-axis.

```
plt.figure(figsize=(12, 5))

plt.plot(df['Name'],df['Physics'],marker="s",label="Name-Phy")
plt.plot(df['Name'],df['Chemistry'],marker="d",label="Name-Chem")
plt.plot(df['Name'],df['English'],marker="o",label="Name-Eng")

plt.legend() # If nothing is mentioned,
            # automatically it will be placed in the
            # best location

plt.grid(axis='x') # Show grid lines
                  # only for x-axis
plt.show()
```



What this does: plt.grid(axis='x') restricts the gridlines to the x-axis only (vertical lines), and calling plt.legend() with no arguments lets Matplotlib automatically pick the "best" location that overlaps the data least.



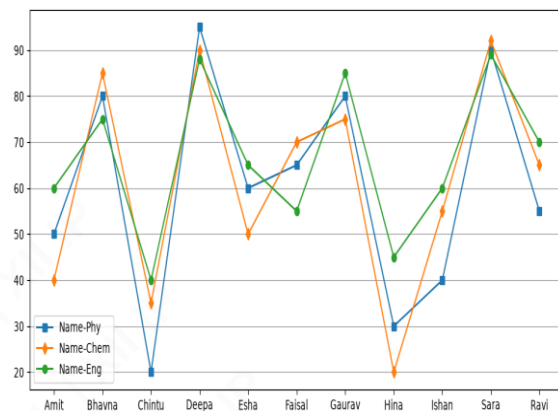


Snippet 30: Gridlines on the Y-Axis Only

Restricting the grid the other way, showing only horizontal lines along the y-axis.

```
plt.figure(figsize=(12, 5))

plt.plot(df['Name'],df['Physics'],marker="s",label="Name-Phy")
plt.plot(df['Name'],df['Chemistry'],marker="d",label="Name-Chem")
plt.plot(df['Name'],df['English'],marker="o",label="Name-Eng")
plt.legend() # If nothing is mentioned, automatically
it will be placed in the best location
plt.grid(axis='y') # Show grid lines only for y-axis
plt.show()
```



What this does: `plt.grid(axis='y')` is the complement of the previous example, drawing only horizontal gridlines (one per y-axis tick) instead of vertical ones.





Bar Graph

A bar graph is drawn using rectangular bars to show how large each value is. The bars can be arranged horizontally or vertically, and their length is proportional to the value they represent.

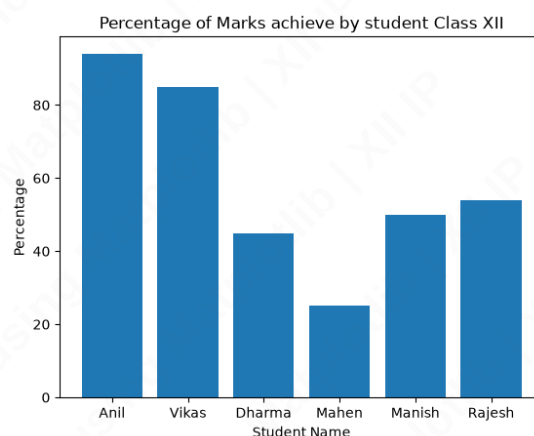
A bar graph makes it easy to compare data between different groups at a glance. It places categories on one axis and a discrete value on the other, so the relationship between the two is immediately visible. Bar graphs are also useful for highlighting large changes in data over time.

A Simple Bar Graph

Snippet 31: Basic Vertical Bar Graph

Plotting student percentages as a set of vertical bars, one per student.

```
import matplotlib.pyplot as plt
import numpy as np
label = ['Anil', 'Vikas', 'Dharma', 'Mahen',
        'Manish', 'Rajesh']
per = [94, 85, 45, 25, 50, 54]
index = np.arange(len(label))
plt.bar(index, per)
plt.xlabel('Student Name')
plt.ylabel('Percentage')
plt.xticks(index, label)
plt.title('Percentage of Marks achieve by student
Class XII')
plt.show()
```

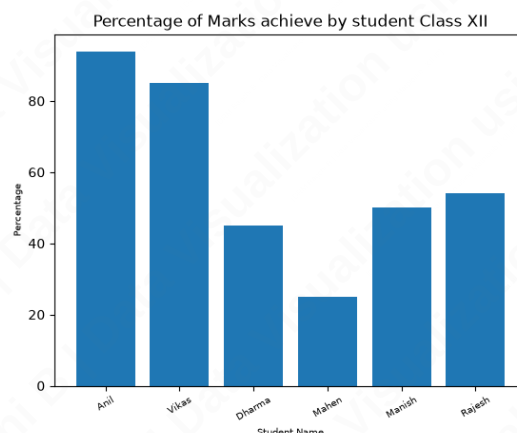


What this does: `np.arange(len(label))` generates one evenly spaced numeric position per student, which `plt.bar()` uses as the x-position of each bar. `plt.xticks(index, label)` then swaps those numeric positions for the actual student names along the x-axis, while `plt.xlabel()`, `plt.ylabel()` and `plt.title()` label the chart.

Snippet 32: Smaller, Rotated Tick Labels

Shrinking and rotating the axis text so longer or more crowded labels stay readable.

```
import matplotlib.pyplot as plt
import numpy as np
label = ['Anil', 'Vikas', 'Dharma', 'Mahen',
        'Manish', 'Rajesh']
per = [94, 85, 45, 25, 50, 54]
index = np.arange(len(label))
plt.bar(index, per)
plt.xlabel('Student Name', fontsize=7)
plt.ylabel('Percentage', fontsize=7)
plt.xticks(index, label, fontsize=7, rotation=30)
plt.title('Percentage of Marks achieve by student
Class XII')
plt.show()
```





What this does: `fontsize=` on `xlabel()/ylabel()` shrinks the axis captions, and passing both `fontsize=` and `rotation=30` to `xticks()` shrinks and tilts the student names, which helps when labels would otherwise overlap.

Custom Bar Appearance

Snippet 33: Custom Bar Color

Filling the bars with a solid color and giving them a contrasting outline.

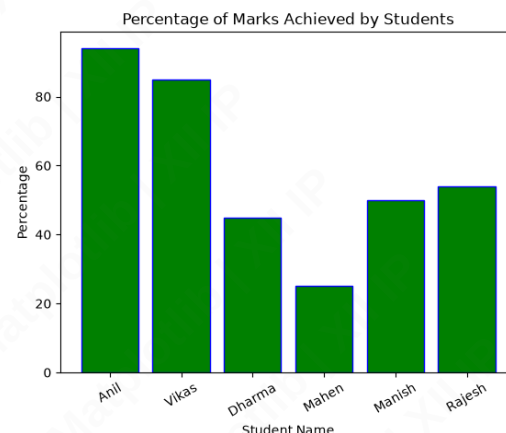
```
import matplotlib.pyplot as plt
import numpy as np

label = ['Anil', 'Vikas', 'Dharma', 'Mahen',
         'Manish', 'Rajesh']
per = [94, 85, 45, 25, 50, 54]

index = np.arange(len(label))

plt.bar(index, per, color="green", edgecolor="blue")

plt.xlabel('Student Name')
plt.ylabel('Percentage')
plt.xticks(index, label, rotation=30)
plt.title('Percentage of Marks Achieved by Students')
plt.show()
```

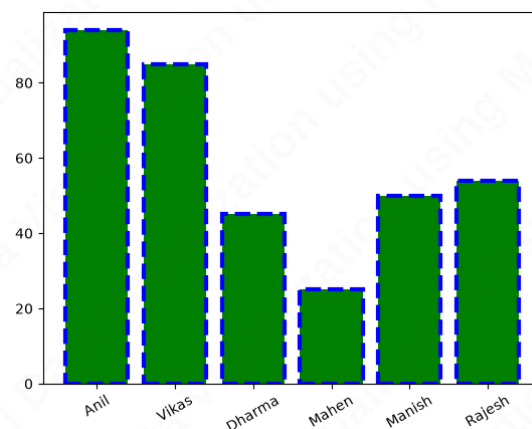


What this does: `color="green"` fills each bar, while `edgecolor="blue"` draws a colored border around every bar so the fill and outline can be styled independently.

Snippet 34: Custom Edge Line Style

Changing the bar outline from a solid border to a dashed one.

```
plt.bar(index, per,
        color="green",
        edgecolor="blue",
        linewidth=3,
        linestyle="--")
plt.xticks(index, label, rotation=30)
plt.show()
```



What this does: `linewidth=3` thickens the bar outlines, and `linestyle="--"` switches the outline from a solid line to a dashed one, giving the bars a more distinct border.





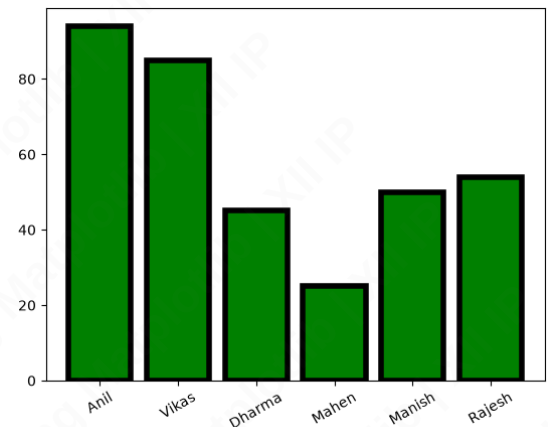
Bar Edge Line Styles

Style Code	Line Style
'-'	Solid
'--'	Dashed
'-.'	Dash-dot
'.'	Dotted

Snippet 35: Custom Edge Line Width

Thickening the bar outline further, using a solid border this time.

```
plt.bar(index, per,
        color="green",
        edgecolor="black",
        linewidth=4)
plt.xticks(index, label, rotation=30)
plt.show()
```

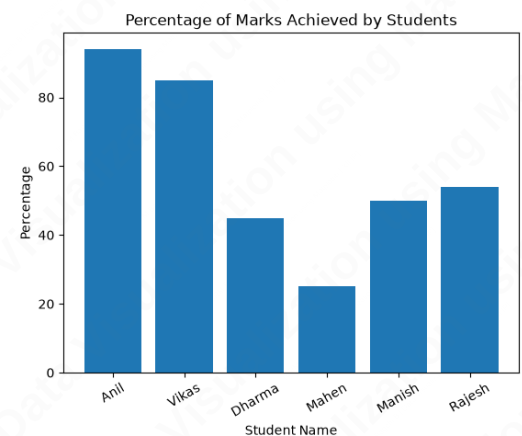


What this does: `linewidth=4` makes the black border around each bar noticeably thicker than the default, while the outline remains solid since no `linestyle` is specified.

Snippet 36: Axis Labels and Custom Title

Returning to the plain bar chart and confirming the axis labels, tick rotation, and title together.

```
plt.bar(index, per)
plt.xlabel('Student Name')
plt.ylabel('Percentage')
plt.xticks(index, label, rotation=30)
plt.title('Percentage of Marks Achieved by Students')
plt.show()
```



What this does: `plt.xlabel()` and `plt.ylabel()` caption the two axes, `plt.xticks(..., rotation=30)` tilts the student names so they do not overlap, and `plt.title()` adds a descriptive heading above the chart.





Adding a Legend

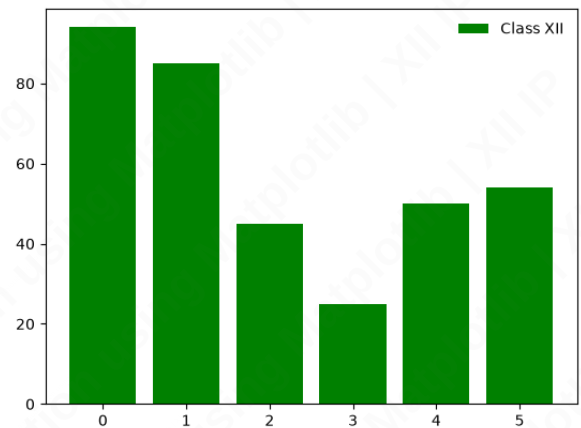
`plt.legend()` adds a legend box that identifies each labelled series. Its position can be fixed with the `loc` parameter, and `frameon` controls whether a border is drawn around the legend box.

Location Value	Position
'upper left'	Top-left corner
'upper right'	Top-right corner
'lower left'	Bottom-left corner
'lower right'	Bottom-right corner
'center'	Center of the axes
'best'	Automatically chosen to overlap the data least

Snippet 37: Legend Without a Frame

Adding a labelled legend to the upper-right corner, with the surrounding frame turned off.

```
plt.bar(index, per, color="green", label="Class XII")  
  
plt.legend(loc='upper right', frameon=False)
```



What this does: `label="Class XII"` on the `bar()` call names this series, and `plt.legend(loc='upper right', frameon=False)` places the legend in the top-right corner without drawing a border box around it.

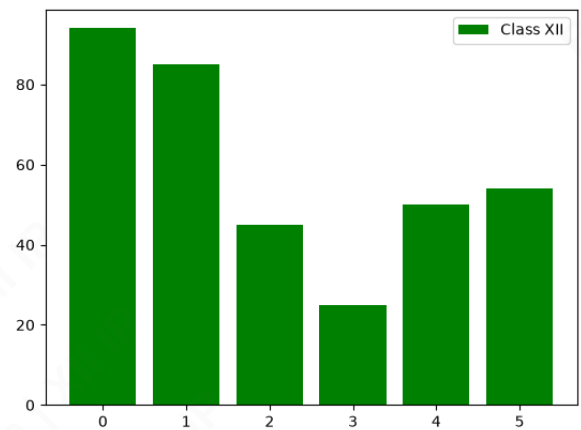
Snippet 38: Legend With a Frame

Letting Matplotlib choose the best legend position automatically, this time with a visible frame.





```
# To show a frame around the legend:  
plt.bar(index, per, color="green", label="Class  
XII")  
plt.legend(loc='best', frameon=True)
```



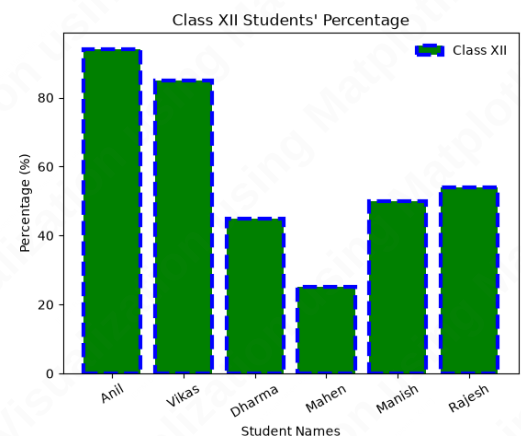
What this does: `loc='best'` lets Matplotlib pick whichever corner overlaps the bars the least, and `frameon=True` (the default) draws a border box around the legend.

Applying All the Customizations

Snippet 39: Combining Color, Style, Labels and Legend

Bringing bar color, edge style, axis labels, title and legend together in a single chart.

```
import matplotlib.pyplot as plt  
import numpy as np  
  
label = ['Anil', 'Vikas', 'Dharma', 'Mahen',  
         'Manish', 'Rajesh']  
per = [94, 85, 45, 25, 50, 54]  
  
index = np.arange(len(label))  
  
plt.bar(index, per,  
        color="green",  
        edgecolor="blue",  
        linewidth=3,  
        linestyle="--",  
        label="Class XII")  
  
plt.xlabel("Student Names", fontsize=10)  
plt.ylabel("Percentage (%)", fontsize=10)  
plt.xticks(index, label, rotation=30)  
  
plt.title("Class XII Students' Percentage")  
plt.legend(loc="upper right", frameon=False)  
  
plt.show()
```



What this does: This snippet layers every customization seen so far onto one chart: a green fill with a dashed blue outline, rotated student-name ticks, sized axis labels, a title, and an unframed legend in the upper-right corner.





Horizontal Bar Graph

Snippet 40: Drawing Bars Horizontally

Switching from vertical to horizontal bars using `barh()` instead of `bar()`.

```
import matplotlib.pyplot as plt
import numpy as np

# Student names
label = ['Anil', 'Vikas', 'Dharma', 'Mahen',
        'Manish', 'Rajesh']

# Percentage marks
per = [94, 85, 45, 25, 50, 54]

# Position of bars
index = np.arange(len(label))

# Horizontal bar graph
plt.barh(index, per,
         color="orange",
         edgecolor="blue",
         linewidth=2,
         linestyle="--",
         label="Class XII")

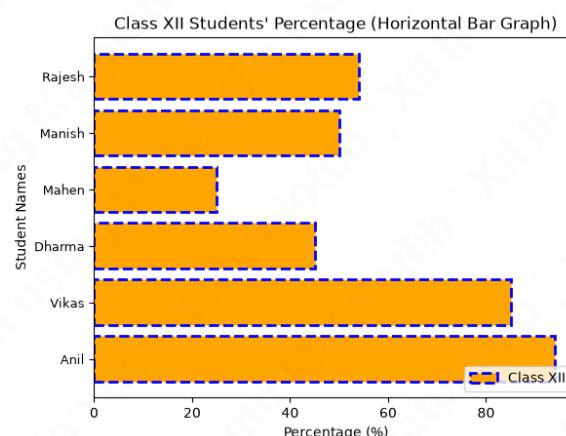
# Labels
plt.xlabel("Percentage (%)", fontsize=10)
plt.ylabel("Student Names", fontsize=10)

# Display student names on Y-axis
plt.yticks(index, label, fontsize=9)

# Title
plt.title("Class XII Students' Percentage
(Horizontal Bar Graph)")

# Legend
plt.legend(loc="lower right", frameon=True)

# Display graph
plt.show()
```



What this does: `plt.barh(index, per)` draws each bar horizontally instead of vertically, so the percentage values now run along the x-axis while `plt.yticks(index, label)` places the student names on the y-axis. The color, edge style, and legend parameters work exactly as they do for `plt.bar()`.





Multiple Bar Plots

A multiple bar plot, also called a grouped bar chart, is used to compare two or more datasets for the same categories. The following examples compare a Half-Yearly and Annual exam, and later a Unit Test, Half-Yearly, and Annual exam, for the same group of students.

Snippet 41: Grouped Bar Chart for Two Datasets

Comparing Half-Yearly and Annual percentages side by side for each student.

```
import matplotlib.pyplot as plt
import numpy as np

# Student names
students = ['Anil', 'Vikas', 'Dharma', 'Mahen',
            'Manish', 'Rajesh']

# Marks in two exams
half_yearly = [85, 78, 60, 45, 70, 68]
annual = [94, 85, 45, 25, 50, 54]

# Position of bars
index = np.arange(len(students))
width = 0.35

# First bar graph
plt.bar(index, half_yearly,
        width=width,
        color='skyblue',
        edgecolor='black',
        label='Half-Yearly')

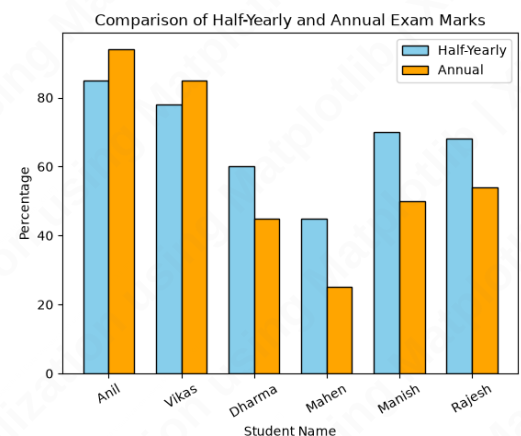
# Second bar graph
plt.bar(index + width, annual,
        width=width,
        color='orange',
        edgecolor='black',
        label='Annual')

# Labels and title
plt.xlabel('Student Name', fontsize=10)
plt.ylabel('Percentage', fontsize=10)
plt.title('Comparison of Half-Yearly and Annual Exam Marks')

# Student names on X-axis
plt.xticks(index + width/2, students, rotation=30)

# Legend
plt.legend(loc='upper right', frameon=True)

# Display graph
plt.show()
```





What this does: `index = np.arange(len(students))` gives one base position per student, and `width = 0.35` sets how wide each bar is. The first `plt.bar()` call draws the Half-Yearly bars at `index`, and the second draws the Annual bars at `index + width`, shifting them to sit right next to the first set. `plt.xticks(index + width/2, students)` then centers each student's name beneath their pair of bars.

Snippet 42: Grouped Bar Chart for Three Datasets

Extending the grouped chart to compare Unit Test, Half-Yearly, and Annual marks together.

```
import matplotlib.pyplot as plt
import numpy as np

# Student names
students = ['Anil', 'Vikas', 'Dharma', 'Mahen',
            'Manish', 'Rajesh']

# Marks in three examinations
unit_test = [78, 72, 55, 40, 62, 58]
half_yearly = [85, 78, 60, 45, 70, 68]
annual = [94, 85, 45, 25, 50, 54]

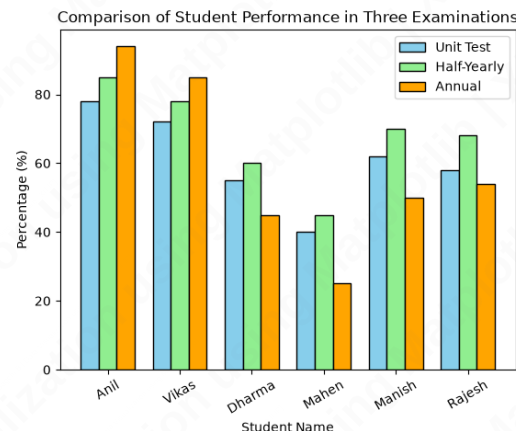
# Position of bars
index = np.arange(len(students))
width = 0.25

# Plot three bar graphs
plt.bar(index - width, unit_test,
        width=width,
        color='skyblue',
        edgecolor='black',
        label='Unit Test')

plt.bar(index, half_yearly,
        width=width,
        color='lightgreen',
        edgecolor='black',
        label='Half-Yearly')

plt.bar(index + width, annual,
        width=width,
        color='orange',
        edgecolor='black',
        label='Annual')

# Labels and title
plt.xlabel('Student Name', fontsize=10)
plt.ylabel('Percentage (%)', fontsize=10)
plt.title('Comparison of Student Performance in
Three Examinations')
```





```
# Display student names on X-axis
plt.xticks(index, students, rotation=30)

# Legend
plt.legend(loc='upper right', frameon=True)

# Display graph
plt.show()
```

What this does: With three datasets, width shrinks to 0.25 and each series is offset from the base position: index - width draws the Unit Test bars on the left, index draws Half-Yearly in the center, and index + width draws Annual on the right, so all three sit side by side for every student without overlapping.

Snippet 43: Grouped Horizontal Bar Chart for Three Datasets

Redrawing the three-exam comparison as horizontal bars using barh().

```
import matplotlib.pyplot as plt
import numpy as np

# Student names
students = ['Anil', 'Vikas', 'Dharma', 'Mahen',
            'Manish', 'Rajesh']

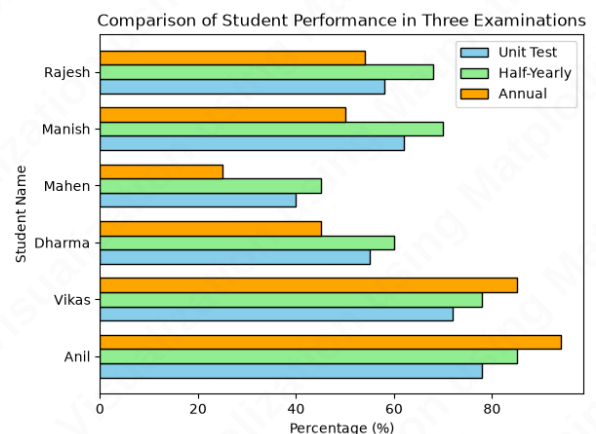
# Marks in three examinations
unit_test = [78, 72, 55, 40, 62, 58]
half_yearly = [85, 78, 60, 45, 70, 68]
annual = [94, 85, 45, 25, 50, 54]

# Position of bars
index = np.arange(len(students))
height = 0.25

# Plot three horizontal bar graphs
plt.barh(index - height, unit_test,
         height=height,
         color='skyblue',
         edgecolor='black',
         label='Unit Test')

plt.barh(index, half_yearly,
         height=height,
         color='lightgreen',
         edgecolor='black',
         label='Half-Yearly')

plt.barh(index + height, annual,
         height=height,
         color='orange',
         edgecolor='black',
```





```
label='Annual')

# Labels and title
plt.xlabel('Percentage (%)', fontsize=10)
plt.ylabel('Student Name', fontsize=10)
plt.title('Comparison of Student Performance in
Three Examinations')

# Display student names on Y-axis
plt.yticks(index, students)

# Legend
plt.legend(loc='upper right', frameon=True)

# Display graph
plt.show()
```

What this does: This mirrors the previous chart with `barh()` in place of `bar()`: height takes the role that width played before, `index - height / index / index + height` stack the three series of bars for each student along the y-axis instead of the x-axis, and `plt.yticks(index, students)` labels them.

bar() vs barh(): Key Differences

The `barh()` function draws horizontal bar charts and is used the same way as `bar()`, just with the category and value axes swapped.

bar() — Vertical Bar Chart	barh() — Horizontal Bar Chart
Bars are drawn vertically.	Bars are drawn horizontally.
Categories appear on the X-axis.	Categories appear on the Y-axis.
Values are shown on the Y-axis.	Values are shown on the X-axis.
Uses width to set bar thickness.	Uses height to set bar thickness.
Uses <code>plt.xticks()</code> for category labels.	Uses <code>plt.yticks()</code> for category labels.





Histogram

A histogram is a graphical representation which organizes a group of data points into user-specified ranges. It provides a visual interpretation of numerical data by showing the number of data points that fall within a specified range of values (“bins”). It is similar to a vertical bar graph, but without gaps between the bars.

A histogram groups data into intervals called bins. Every value in the dataset is placed into exactly one of these intervals, and the height of the corresponding bar equals the number of values that fall inside it. For example, with bins=[0,10,20,30,40,50,60], six intervals are created: 0-10, 10-20, 20-30, 30-40, 40-50, and 50-60.

Simple Histogram

Snippet 44: Simple Histogram

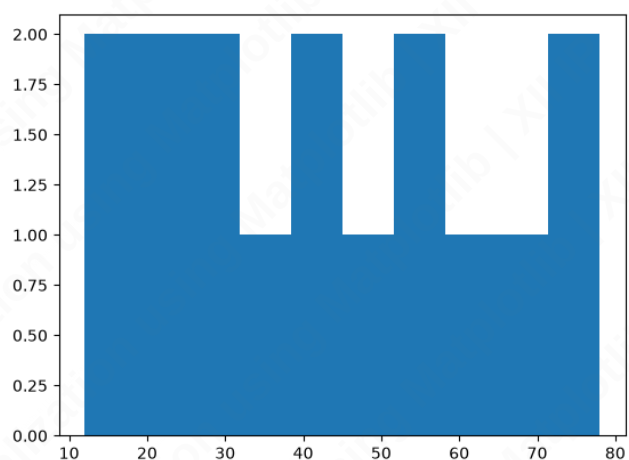
Plotting a basic histogram from a list of student marks without specifying bins.

```
import matplotlib.pyplot as plt

marks = [12, 18, 22, 24, 27, 31, 35, 39, 42,
         48, 53, 56, 61, 65, 72, 78]

plt.hist(marks)

plt.show()
```



What this does: The list marks holds the raw observations. Since the bins parameter is not specified, plt.hist() automatically chooses suitable intervals. The height of each bar shows how many values fall inside that interval, and plt.show() renders the figure.





Specifying Bins

Snippet 45: Specifying Bins

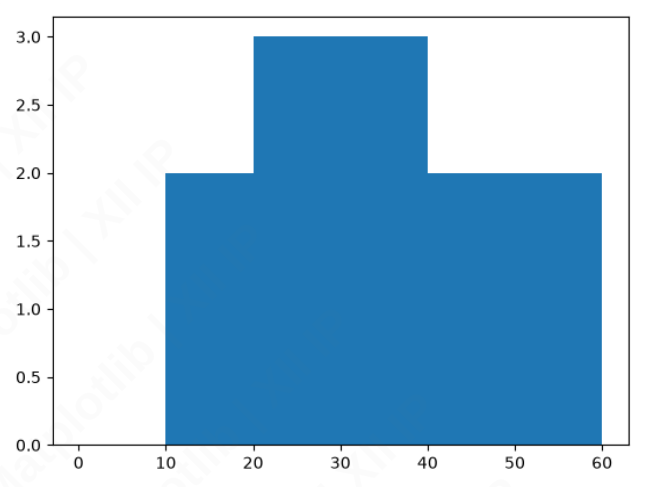
Controlling the exact interval boundaries by passing a list to the bins parameter.

```
import matplotlib.pyplot as plt

marks = [12,18,22,24,27,31,35,39,42,48,53,56]

plt.hist(marks,
        bins=[0,10,20,30,40,50,60])

plt.show()
```



What this does: Passing `bins=[0,10,20,30,40,50,60]` manually fixes six intervals of width 10. Every value in `marks` is placed into exactly one of these intervals, and the bar height equals the frequency (count) of values in that interval.

Histogram with Weights

Snippet 46: Histogram with Weights

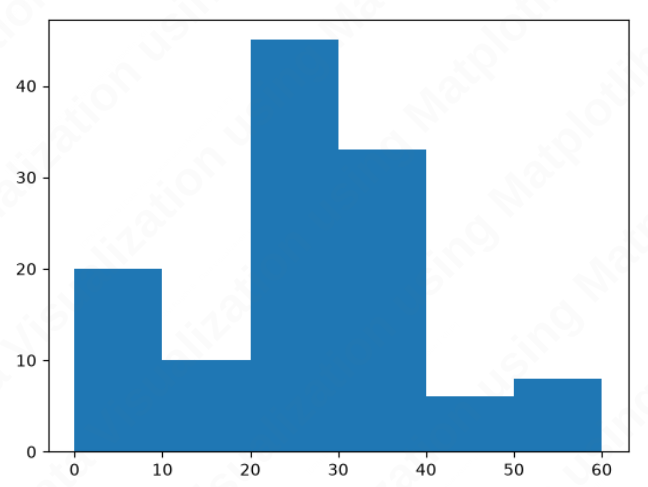
Using the weights parameter so each position contributes a custom value instead of a simple count of 1.

```
import matplotlib.pyplot as plt

positions = [5,15,25,35,45,55]
weights = [20,10,45,33,6,8]

plt.hist(positions,
        bins=[0,10,20,30,40,50,60],
        weights=weights)

plt.show()
```



What this does: The `weights` parameter replaces the default frequency count with a custom value for each entry in `positions`. Each position falls into a bin, and the bar height becomes the corresponding weight rather than a plain count. The two lists must be of equal length, or Python raises an error.





Snippet 47: Multiple Values in the Same Bin

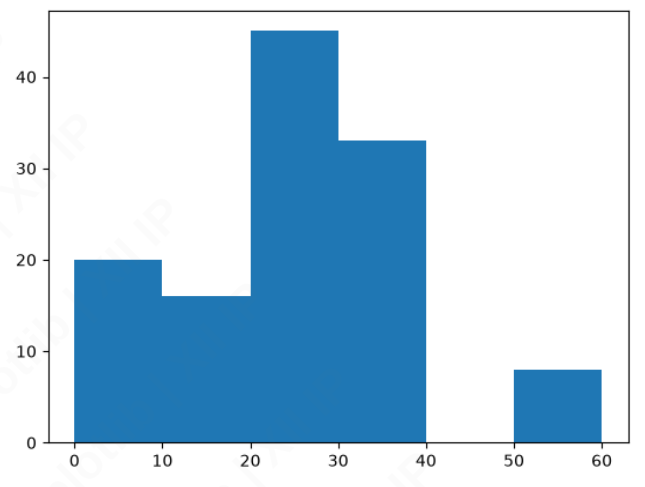
Observing how weights combine when two different positions land in the same bin.

```
import matplotlib.pyplot as plt

positions = [5,15,25,35,15,55]
weights = [20,10,45,33,6,8]

plt.hist(positions,
          bins=[0,10,20,30,40,50,60],
          weights=weights)

plt.show()
```



What this does: Here the value 15 appears twice in positions, with weights 10 and 6. Both occurrences fall in the 10-20 interval, so Matplotlib adds their weights together, giving a bar height of 16 for that bin. This demonstrates that weights accumulate within the same interval.

Customizing Histogram

Snippet 48: Changing Bar Color

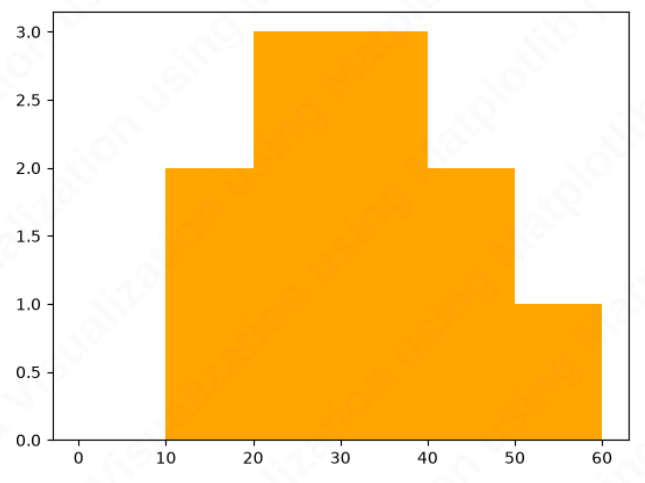
Customizing the fill color of the histogram bars with the color parameter.

```
import matplotlib.pyplot as plt

marks = [12,18,22,24,27,31,35,39,42,48,53]

plt.hist(marks,
          bins=[0,10,20,30,40,50,60],
          color="orange")

plt.show()
```



What this does: The color parameter sets the fill color of every bar; here it is set to "orange" instead of Matplotlib's default blue.

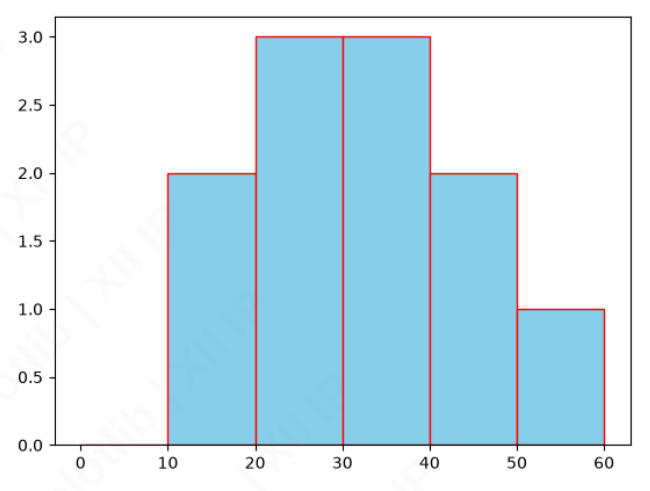




Snippet 49: Changing Edge Color

Adding a distinct border color to each bar using the `edgecolor` parameter.

```
plt.hist(marks,  
        bins=[0,10,20,30,40,50,60],  
        color="skyblue",  
        edgecolor="red")  
plt.show()
```

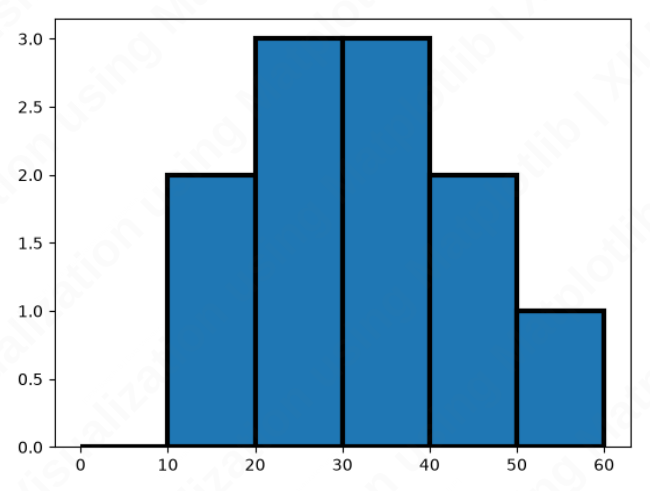


What this does: The `edgecolor` parameter draws a colored outline around each bar (here, red), which makes bar boundaries easier to see against the "skyblue" fill.

Snippet 50: Changing Edge Width

Making the bar borders thicker with the `linewidth` parameter.

```
plt.hist(marks,  
        bins=[0,10,20,30,40,50,60],  
        edgecolor="black",  
        linewidth=3)  
plt.show()
```



What this does: The `linewidth` parameter controls the thickness of the bar borders set by `edgecolor`; a larger value produces a bolder outline.

Snippet 51: Making Bars Transparent

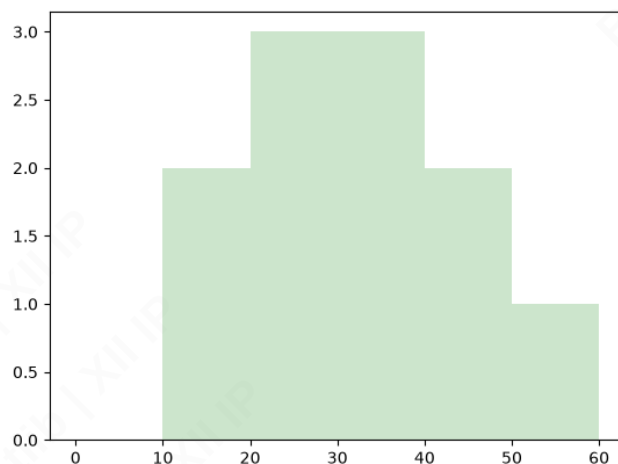
Controlling how see-through the bars are using the `alpha` parameter.





Data Visualization using Matplotlib

```
plt.hist(marks,  
        bins=[0,10,20,30,40,50,60],  
        color="green",  
        alpha=0.2)  
plt.show()
```

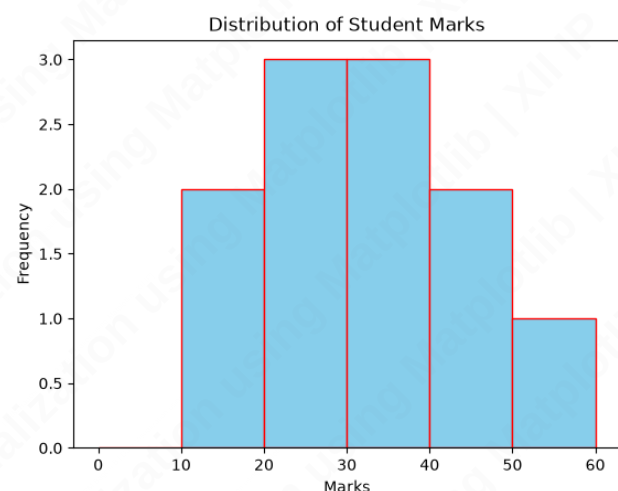


What this does: The alpha parameter sets bar transparency, ranging from 0 (completely transparent) to 1 (completely opaque). A value of 0.2 makes the green bars mostly see-through.

Snippet 52: Adding Title, X-Label and Y-Label

Labeling the chart with a title and axis names for clarity.

```
plt.hist(marks,  
        bins=[0,10,20,30,40,50,60],  
        color="skyblue",  
        edgecolor="red")  
plt.xlabel("Marks")  
plt.ylabel("Frequency")  
plt.title("Distribution of Student Marks")  
plt.show()
```

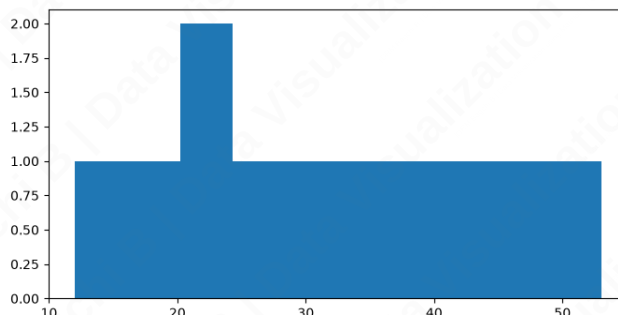


What this does: plt.xlabel() and plt.ylabel() name the horizontal and vertical axes, while plt.title() adds a heading above the chart, making the histogram self-explanatory.

Snippet 53: Changing Figure Size

Resizing the overall plot area before drawing the histogram.

```
plt.figure(figsize=(8,4))  
plt.hist(marks)  
plt.show()
```



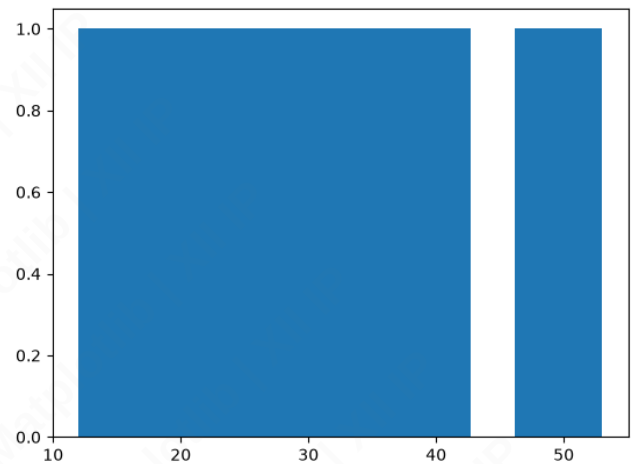


What this does: `plt.figure(figsize=(8,4))` must be called before `plt.hist()`; it sets the figure width and height in inches, here producing a wider, shorter chart.

Snippet 54: Changing Bin Width - Small Bins

Using a larger bin count to reveal finer detail in the distribution.

```
plt.hist(marks,bins=12)  
plt.show()
```

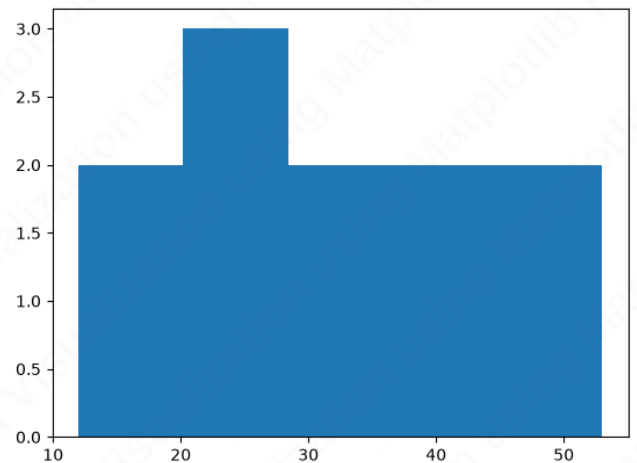


What this does: Setting `bins=12` asks Matplotlib to divide the data range into 12 equal-width intervals instead of its default count, producing narrower bars and more detail.

Snippet 55: Changing Bin Width - Large Bins

Using a smaller bin count to get a simpler overview of the distribution.

```
plt.hist(marks,bins=5)  
plt.show()
```



What this does: Setting `bins=5` produces fewer, wider bars. Smaller bin counts give a simpler overview, while larger bin counts (as in the previous snippet) give more detail.

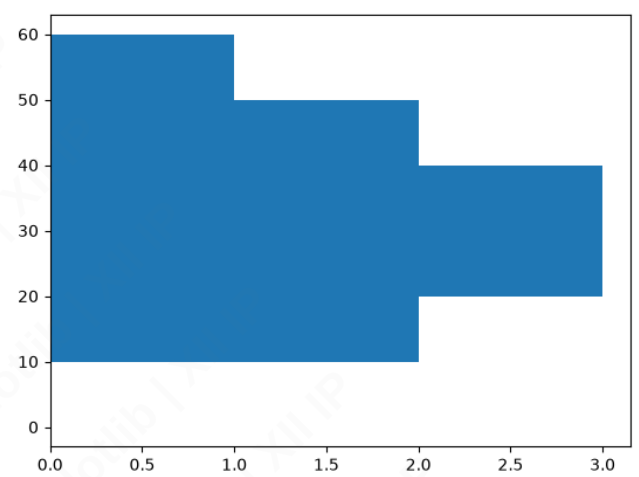




Snippet 56: Horizontal Histogram

Flipping the chart axis using the orientation parameter.

```
plt.hist(marks,  
         bins=[0,10,20,30,40,50,60],  
         orientation="horizontal")  
plt.show()
```

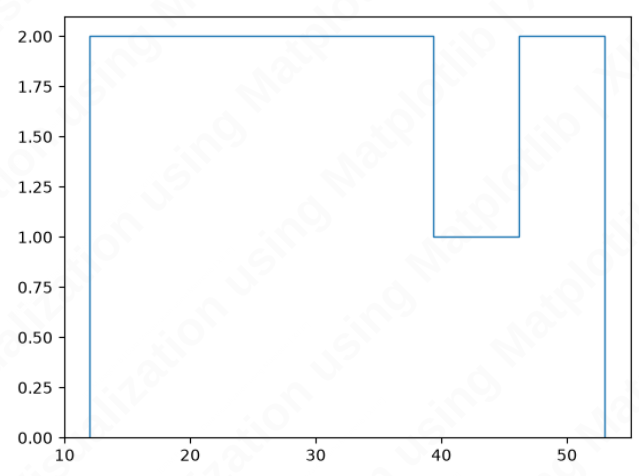


What this does: The orientation parameter accepts "vertical" (default) or "horizontal"; setting it to "horizontal" draws bars sideways along the y-axis.

Snippet 57: Histogram Style - Step

Drawing the histogram outline only, using the histtype parameter.

```
plt.hist(marks,  
         bins=6,  
         histtype="step")  
plt.show()
```



What this does: The histtype parameter changes how bars are rendered. "step" draws only the outline of each bar, without fill. Other available styles are bar, barstacked, and stepfilled.

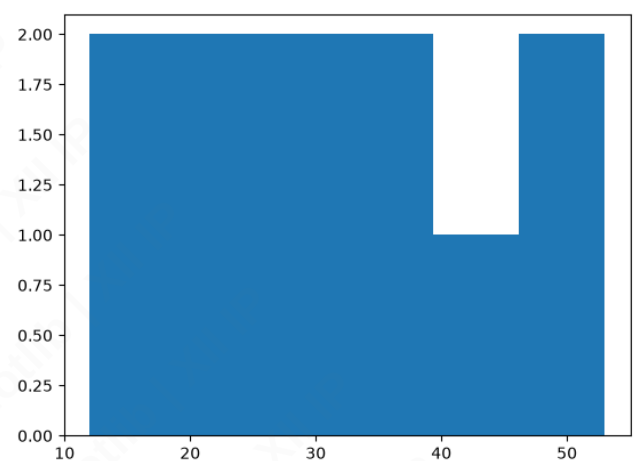




Snippet 58: Histogram Style - Stepfilled

Combining the step outline with a solid fill using `histtype="stepfilled"`.

```
plt.hist(marks,  
         bins=6,  
         histtype="stepfilled")  
plt.show()
```

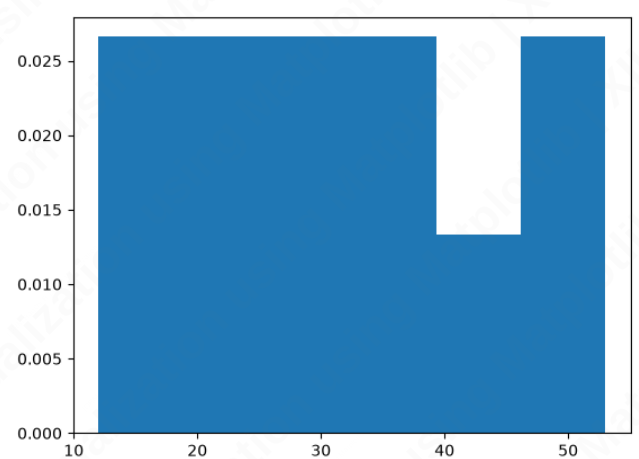


What this does: "stepfilled" draws the same outline style as "step" but fills the area beneath it, giving a solid, continuous silhouette.

Snippet 59: Displaying Density Instead of Frequency

Normalizing the histogram so it represents a probability density.

```
plt.hist(marks,  
         bins=6,  
         density=True)  
plt.show()
```



What this does: Setting `density=True` rescales bar heights so the total area under the histogram equals 1, which is useful when comparing the shape of the distribution to a probability curve.

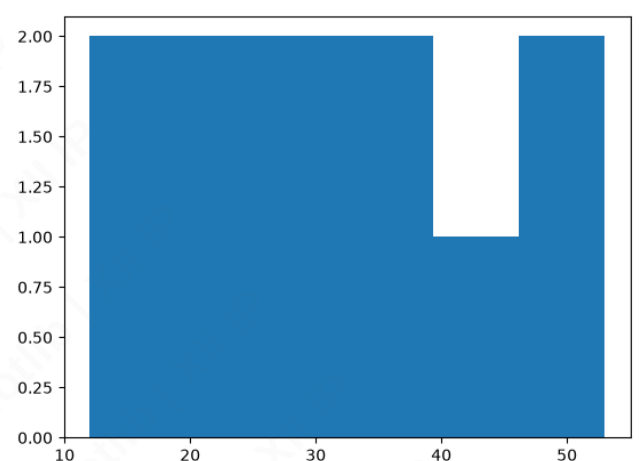




Snippet 60: Aligning the Bars

Controlling bar placement relative to bin edges with the align parameter.

```
plt.hist(marks,  
         bins=6,  
         align="mid")  
plt.show()
```

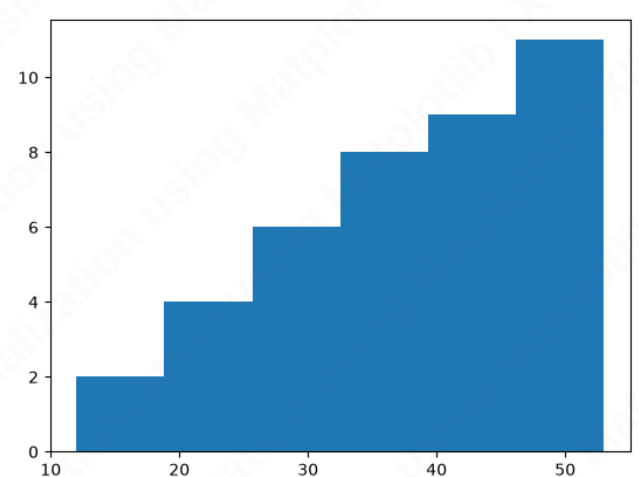


What this does: The align parameter positions each bar relative to its bin edges. It accepts left, mid (used here), or right.

Snippet 61: Cumulative Histogram

Plotting running totals instead of individual bin frequencies.

```
plt.hist(marks,  
         bins=6,  
         cumulative=True)  
plt.show()
```



What this does: Setting cumulative=True makes each bar represent the cumulative frequency up to and including that bin, so the bars steadily increase from left to right.

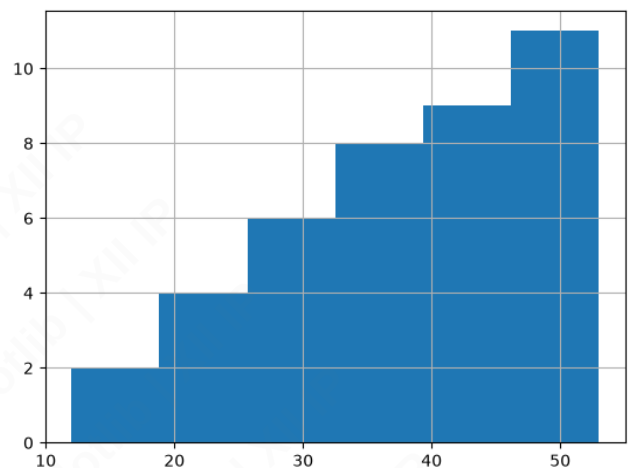




Snippet 62: Adding Grid Lines

Overlaying gridlines on the cumulative histogram to improve readability.

```
plt.hist(marks,  
         bins=6,  
         cumulative=True)  
plt.grid(True)  
plt.show()
```

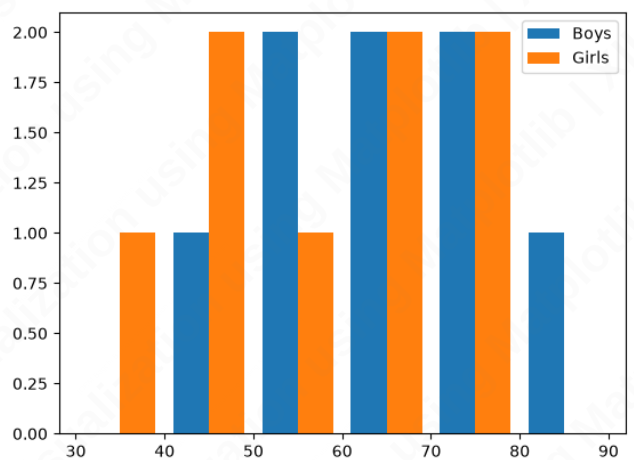


What this does: `plt.grid(True)` draws horizontal and vertical gridlines across the plot area, making it easier to read bar heights precisely.

Snippet 63: Plotting Multiple Histograms

Comparing two groups (boys and girls) on the same axes with a legend.

```
import matplotlib.pyplot as plt  
  
boys = [42,50,52,60,65,70,75,80]  
girls = [38,45,49,55,60,66,72,78]  
  
plt.hist([boys,girls],  
         bins=[30,40,50,60,70,80,90],  
         label=["Boys","Girls"])  
  
plt.legend()  
plt.show()
```



What this does: Passing a list of lists, `[boys,girls]`, plots side-by-side bars for each dataset within every bin. The `label` parameter names each dataset, and `plt.legend()` displays those names in a legend box so the two groups can be told apart.





Snippet 64: Complete Example

Combining figure size, colors, labels, title and gridlines into one polished histogram.

```
import matplotlib.pyplot as plt

marks = [12,18,22,24,27,31,35,39,42,48,53,56]

plt.figure(figsize=(8,5))

plt.hist(
    marks,
    bins=[0,10,20,30,40,50,60],
    color="skyblue",
    edgecolor="black",
    linewidth=2,
    alpha=0.8
)

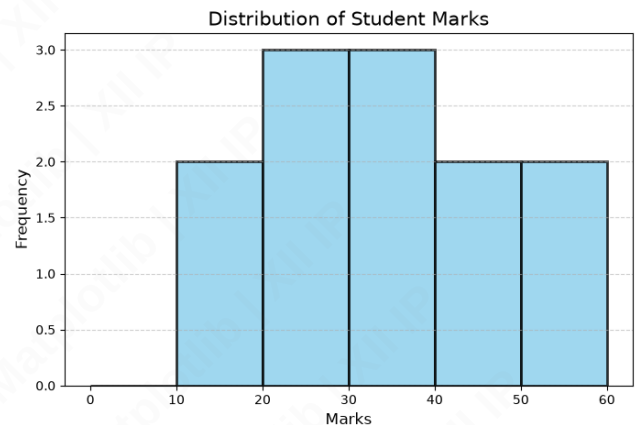
plt.title("Distribution of Student Marks", fontsize=15)

plt.xlabel("Marks", fontsize=12)

plt.ylabel("Frequency", fontsize=12)

plt.grid(axis="y", linestyle="--", alpha=0.6)

plt.show()
```



What this does: This example combines several parameters covered earlier: a custom figsize, fill and edge colors, transparency, a titled and labeled chart, and dashed y-axis gridlines restricted with axis="y", producing a presentation-ready histogram.

Saving the Figure

Snippet 65: Saving the Figure

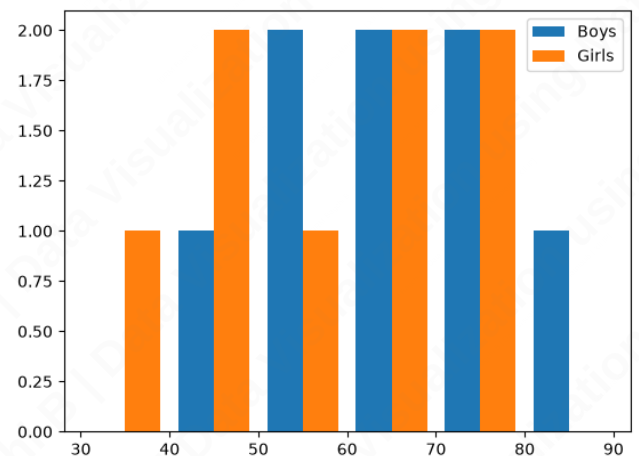
Writing the chart to a file on disk using `savefig()` before calling `show()`.

```
import matplotlib.pyplot as plt

boys = [42,50,52,60,65,70,75,80]
girls = [38,45,49,55,60,66,72,78]

plt.hist([boys, girls],
        bins=[30,40,50,60,70,80,90],
        label=["Boys", "Girls"])

plt.legend()
plt.savefig('Histogram.png') # Should be
                             # placed before plt.show()
plt.show()
```



What this does: `plt.savefig('Histogram.png')` writes the figure to the current working directory. It must be called before `plt.show()`, otherwise an empty figure may be saved. Plots can be saved in pdf, svg, png, or jpg formats, and the method works the same way for every chart type.



**Frequently Used Parameters of plt.hist()**

Parameter	Purpose
x	Data values
bins	Defines the intervals (bins)
weights	Assigns custom weights to observations
color	Fill color of bars
edgecolor	Color of bar borders
linewidth	Thickness of borders
alpha	Transparency of bars
histtype	Histogram style (bar, step, etc.)
orientation	Vertical or horizontal histogram
density	Normalizes histogram so the total area equals 1
cumulative	Shows cumulative frequencies
align	Alignment of bars within bins
label	Label used in the legend
rwidth	Relative width of bars (0 to 1)

Saving Plots - Summary

For future use, plots can be saved with the `savefig()` method, in formats such as pdf, svg, png, or jpg. The figure is written to the current working directory, and this method applies to every chart type covered in this course.

